Week 1

OWASP Top 10: Research on Five Critical Web Application Vulnerabilities

Vortex Tech- Cyber Security Internship Track

Week 1 of 4- Beginner

Submitted to:

Vortex tech

Prepared by:

Om bajracharya

Security analyst

Table of Contents

Introduction

The OWASP (Open Web Application Security Project) Top 10 is the industry-standard reference for the most critical security risks facing web applications today. This document covers five vulnerabilities from the current list (OWASP Top 10:2025), explained in plain language, along with how attackers exploit them, a real-world example of each, and practical prevention steps developers can take.

Objective:

Build a strong theoretical foundation by researching and documenting the most common web application security vulnerabilities, as defined by the OWASP Top 10 the industry-standard reference every security professional knows.

The five vulnerabilities covered here are:

1. Broken Access Control
2. Security Misconfiguration
3. Cryptographic Failures
4. Injection
5. Authentication Failures

1. Broken Access Control (A01:2025)

Background?

Access control is the set of rules that decide who is allowed to see or do what inside an application for example, making sure a regular user can't view another user's private data, or that only admins can access admin settings. "Broken" access control means those rules aren't properly enforced, so people can reach data or actions they were never supposed to touch.

How an Attacker Could Exploit It?

A very common form is called an Insecure Direct Object Reference (IDOR). Imagine a banking app where your account statement is loaded from a URL like `bank.com/statement?id=1001`. If the app doesn't check whether the logged-in user actually owns account 1001, an attacker can simply change the number in the URL ,`id=1002`, `id=1003`, and so on ,and pull up other customers' statements. No hacking tools required, just guessing or incrementing a number.

Real-World Example

In 2018, the United States Postal Service's "Informed Visibility" web tool had an authentication flaw that let any logged-in user query the system and retrieve account details for roughly 60 million other users, simply by manipulating the request. A security researcher discovered and reported the issue before it could be widely abused, but it illustrates how a missing ownership checks on an otherwise "authenticated" feature can expose massive amounts of data.

Prevention

Developers should enforce access control checks on the server side for every request, not just in the app's front-end menus every request should verify the specific record belongs to (or is permitted for) the requesting user, rather than assuming that being logged in is enough.

2. Security Misconfiguration (A02:2025)

Background

This vulnerability isn't a single flaw but a category of mistakes in how software, servers, and cloud services are set up ,things like leaving default passwords in place, leaving debug features turned on in production, or leaving cloud storage open to the public. As applications increasingly run on cloud infrastructure and containers, this category has become one of the most common causes of breaches.

How an Attacker Could Exploit It

Attackers routinely scan the internet looking for cloud storage buckets, admin panels, or servers that were never properly locked down. If a company stores customer files in a cloud storage bucket and forgets to restrict who can read it, anyone with the link ,or anyone running an automated scanner ,can browse and download the contents without needing a password at all.

Real-World Example

Multiple large-scale breaches have stemmed from exposed, misconfigured cloud storage. In several well-documented cases, companies and their third-party vendors left Amazon S3 storage buckets set to public access, exposing millions of customer records,including names, addresses, and account details ,to anyone who found the link, with no hacking or password-cracking needed at all.

Prevention

Teams should treat configuration as seriously as code: remove default accounts and credentials, disable unnecessary features and debug endpoints before going live, and regularly audit cloud

storage and infrastructure settings (ideally with automated scanning tools) to catch anything left publicly accessible.

3. Cryptographic Failures (A04:2025)

Background

This covers situations where sensitive data ,passwords, credit card numbers, health records ,isn't properly protected with encryption, or is protected with weak/outdated encryption methods. It used to be called "Sensitive Data Exposure," but OWASP renamed it to point at the actual root cause: weak or missing cryptography.

How an Attacker Could Exploit It

If a company stores passwords in plain text, or encrypted with an outdated or weak method, an attacker who manages to steal the database doesn't need to "hack" anything further ,they can read the passwords directly, or crack them very quickly with modern hardware. The damage compounds because people often reuse the same password across multiple sites.

Real-World Example

In Adobe's 2013 breach, attackers stole a database containing credentials for over 150 million user accounts. Rather than being properly hashed (a one-way, hard-to-reverse process), the passwords had been encrypted with a method that let researchers spot patterns ,for example, spotting which users shared the same password ,because identical passwords produced identical encrypted output. This significantly weakened the protection the encryption was supposed to provide.

Prevention

Sensitive data should be encrypted both at rest (in storage) and in transit (over the network, via HTTPS/TLS). Passwords specifically should never be encrypted in a reversible way ,they should be hashed using a modern, purpose-built algorithm designed to resist cracking (such as bcrypt or Argon2), with a unique random "salt" added per password so identical passwords don't produce identical results.

4. Injection (A05:2025)

Background

Injection happens when an application takes input from a user and passes it directly into a command ,most often a database query ,without properly checking or filtering it first. This lets an

attacker sneak their own instructions into that command. SQL Injection (targeting databases) is the most famous variant.

How an Attacker Could Exploit It

Picture a login form that builds a database query by directly pasting in whatever the user types, like: `SELECT * FROM users WHERE username = '[input]' AND password = '[input]'`. If the app doesn't sanitize input, an attacker could type something like `' OR '1'='1` into the username field. Because `'1'='1'` is always true, this can trick the database into returning a valid user record and logging the attacker in without knowing any real password ,or in more advanced cases, let them read, modify, or delete data across the entire database

Real-World Example

The 2008 Heartland Payment Systems breach is one of the largest SQL injection attacks in history. Attackers found a poorly coded, years-old web login page that failed to properly filter user input, and used a SQL injection attack to slip malicious database commands through it. That initial foothold let them plant data-stealing malware on Heartland's payment processing network, ultimately exposing over 130 million credit and debit card numbers and costing the company tens of millions of dollars in remediation and settlements.

Prevention

The most effective defense is using parameterized queries (also called prepared statements), which treat user input strictly as data ,never as executable code ,no matter what characters it contains. This structurally prevents the "trick" injection relies on, and is considered a baseline requirement for any application that touches a database.

5. Authentication Failures (A07:2025)

Background

This covers weaknesses in how an application verifies that users are who they claim to be ,things like allowing weak passwords, not limiting failed login attempts, or failing to protect against automated login attacks. Even if everything else about an app is secure, a weak front door undermines it all.

How an Attacker Could Exploit It

One common technique is "credential stuffing": attackers take huge lists of usernames and passwords leaked from unrelated past breaches and automatically try them against a different website, betting that some users reused the same password. If the target site doesn't limit login attempts or detect this automated pattern, attackers can silently take over thousands of accounts.

Real-World Example

In 2020, as Zoom's user base exploded during the shift to remote work, attackers used credential stuffing ,reusing passwords leaked from older, unrelated breaches ,to compromise around 500,000 Zoom accounts, which were then sold or given away on hacker forums. The accounts themselves hadn't been "hacked" directly; the weakness was that reused, previously-leaked passwords still worked and there wasn't enough friction (like rate-limiting or multi-factor authentication) to stop automated attempts.

Prevention

Developers should enforce multi-factor authentication (MFA) wherever possible, rate-limit and lock out repeated failed login attempts, and check new passwords against known-breached password lists at sign-up time so users can't pick a password that's already circulating publicly.

Personal Reflection

Of the five, **Security Misconfiguration** surprised me the most. I expected the scariest vulnerabilities to require real technical skill ,writing exploit code, cracking encryption, or finding some obscure bug. Instead, some of the largest breaches in history came down to someone simply forgetting to flip a permission switch on a cloud storage bucket. It's a reminder that in security, the biggest risks aren't always the most "hackable" ones ,they're often the most avoidable ones, caused by a missed checklist item rather than a sophisticated attacker. That makes it both less intimidating and more urgent: good habits and configuration reviews can prevent a huge share of real-world breaches.

References

- OWASP Top 10:2025 ,https://owasp.org/Top10/2025/

- OWASP Top 10:2025 ,A01 Broken Access Control

- OWASP Top 10:2025 ,A02 Security Misconfiguration

- OWASP Top 10:2025 ,A04 Cryptographic Failures

- OWASP Top 10:2025 ,A05 Injection

- OWASP Top 10:2025 ,A07 Authentication Failures